

2.10

No. of Connected Components

0	1	1	0
0	1	1	0
0	0	1	0
0	0	0	0
1	1	0	1

no. of islands → ?

3 starting nodes

↓
3 islands



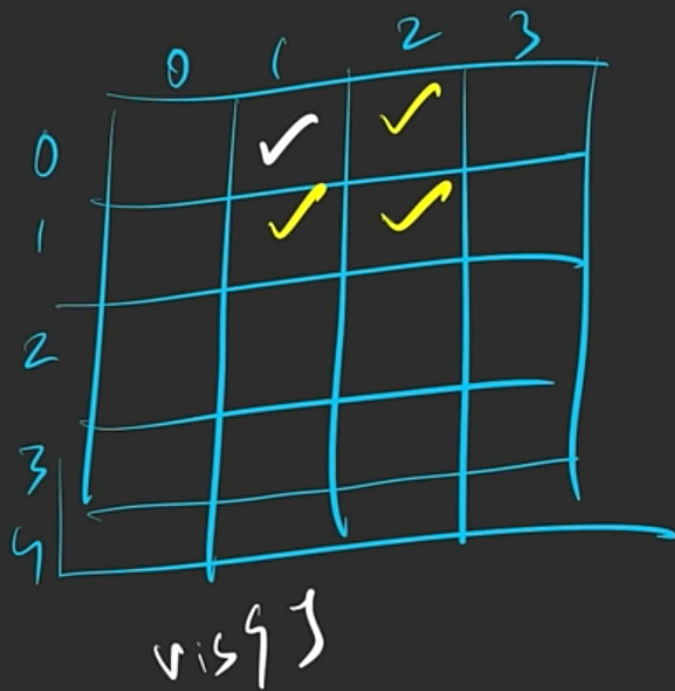
2.10

	0	1	2	3
0	0	1	1	0
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	1	1	0	1

Starting $\rightarrow \{0, 1\}$

1, 1
1, 2
0, 2
3, 3
0

2.10



2.10

No. of Connected Components

BFSStarting $\rightarrow (4, 0)$

	0	1	2	3
0	0	✓	✓	0
1	0	✓	✓	0
2	0	0	✓	0
3	0	0	0	0
4	✓	✓	0	1

2	2
1	1
1	1
0	2
3	1

0

1

2.10

No. of Connected Components

BFSShurong $\rightarrow$ {4,3}

	0	1	2	3
0	0	✓	✓	0
1	0	✓	✓	0
2	0	0	✓	0
3	0	0	0	0
4	✓	✓	0	1

2	2
1	1
1	2
0	2
3	1

0

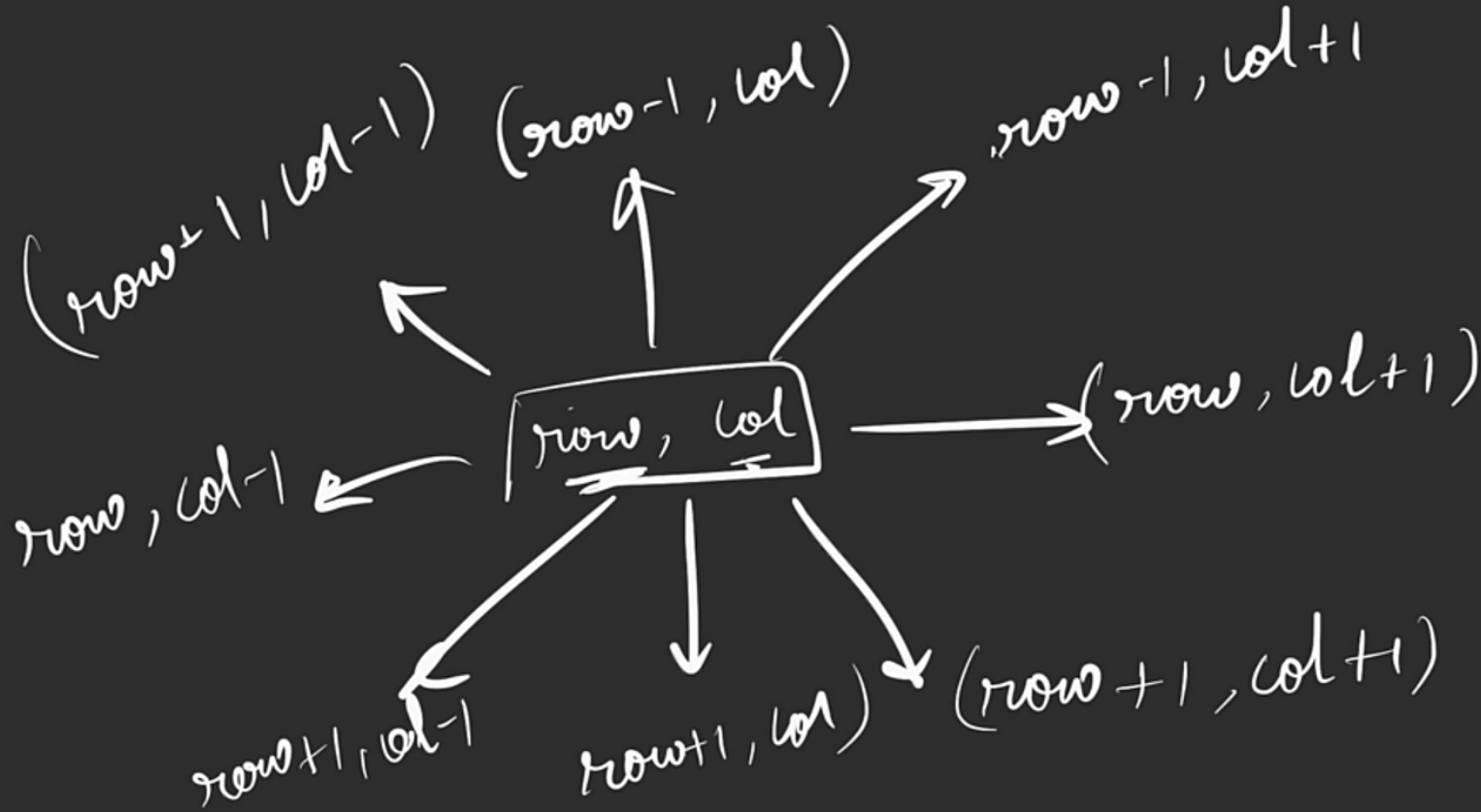
1



2.10

```
for (row → (0 - n))  
{  
    for (col → (0 - m))  
    {  
        if (matrix[row][col] == 1)  
        {  
            dfs(row, col)  
            count++;  
        }  
    }  
}
```





2.10

```
for (del row  $\rightarrow$  -1 to +1)  
{  
    for (del col  $\rightarrow$  -1 to +1)  
    {  
        neigh row = row + del row  
        neigh col = col + del col;  
    }  
}
```



G-8. Number of Islands | Number of Connected Components in Matrix | C++ | Java

```
class Pair {
2.1 int first;
    int second;
    public Pair(int first, int second) {
        this.first = first;
        this.second = second;
    }
}

class Solution {
    private void bfs(int ro, int co, int[][] vis, char[][] grid) {
        vis[ro][co] = 1;
        Queue<Pair> q = new LinkedList<Pair>();
        q.add(new Pair(ro, co));
        int n = grid.length;
        int m = grid[0].length;

        while(!q.isEmpty()) {
            int row = q.peek().first;
            int col = q.peek().second;
            q.remove();

            // traverse in the neighbours and mark them if its a land
            for(int delrow = -1; delrow<=1;delrow++) {
                for(int delcol = -1; delcol<= 1; delcol++) {
                    int nrow = row + delrow;
                    int ncol = col + delcol;
                    if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
                    && grid[nrow][ncol] == '1' && vis[nrow][ncol] == 0) {
                        vis[nrow][ncol] = 1;
                        q.add(new Pair(nrow, ncol));
                    }
                }
            }
        }

        // Function to find the number of islands.
        public int numIslands(char[][] grid) {
            int n = grid.length;
            int m = grid[0].length;
            int[][] vis = new int[n][m];
            int cnt = 0;
            for(int row = 0; row < n; row++) {
                for(int col = 0; col < m; col++) {
                    if(vis[row][col] == 0 && grid[row][col] == '1') {
                        cnt++;
                        bfs(row, col, vis, grid);
                    }
                }
            }
            return cnt;
        }
    }
}
```

```
1 // } Driver Code Ends
6 class Solution {
7     private:
8     void bfs(int row, int col, vector<vector<int>> &vis, vector<vector<cha
9         vis[row][col] = 1;
10        queue<pair<int,int>> q;
11        q.push({row, col});
12        int n = grid.size();
13        int m = grid[0].size();
14
15        while(!q.empty()) {
16            int row = q.front().first;
17            int col = q.front().second;
18            q.pop();
19
20            // traverse in the neighbours and mark them if its a land
21            for(int delrow = -1; delrow<=1;delrow++) {
22                for(int delcol = -1; delcol <= 1; delcol++) {
23                    int nrow = row + delrow;
24                    int ncol = col + delcol;
25                    if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
26                    && grid[nrow][ncol] == '1' && !vis[nrow][ncol]) {
27                        vis[nrow][ncol] = 1;
28                        q.push({nrow, ncol});
29                    }
30                }
31            }
32        }
33    }
34    public:
35        // Function to find the number of islands.
36        int numIslands(vector<vector<char>>& grid) {
37            int n = grid.size();
38            int m = grid[0].size();
```



Compile & Run

G-8. Number of Islands | Number of Connected Components in Matrix | C++ | Java

```
class Pair {
2.1 int first;
    int second;
    public Pair(int first, int second) {
        this.first = first;
        this.second = second;
    }
}

class Solution {
    private void bfs(int ro, int co, int[][] vis, char[][] grid) {
        vis[ro][co] = 1;
        Queue<Pair> q = new LinkedList<Pair>();
        q.add(new Pair(ro, co));
        int n = grid.length;
        int m = grid[0].length;

        while(!q.isEmpty()) {
            int row = q.peek().first;
            int col = q.peek().second;
            q.remove();

            // traverse in the neighbours and mark them if its a land
            for(int delrow = -1; delrow<=1;delrow++) {
                for(int delcol = -1; delcol <= 1; delcol++) {
                    int nrow = row + delrow;
                    int ncol = col + delcol;
                    if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
                    && grid[nrow][ncol] == '1' && vis[nrow][ncol] == 0) {
                        vis[nrow][ncol] = 1;
                        q.add(new Pair(nrow, ncol));
                    }
                }
            }
        }
    }

    // Function to find the number of islands.
    public int numIslands(char[][] grid) {
        int n = grid.length;
        int m = grid[0].length;
        int[][] vis = new int[n][m];
        int cnt = 0;
        for(int row = 0; row < n; row++) {
            for(int col = 0; col < m; col++) {
                if(vis[row][col] == 0 && grid[row][col] == '1') {
                    cnt++;
                    bfs(row, col, vis, grid);
                }
            }
        }
        return cnt;
    }
}
```

```
16         int row = q.front().first;
17         int col = q.front().second;
18         q.pop();
19
20         // traverse in the neighbours and mark them if its a land
21         for(int delrow = -1; delrow<=1;delrow++) {
22             for(int delcol = -1; delcol <= 1; delcol++) {
23                 int nrow = row + delrow;
24                 int ncol = col + delcol;
25                 if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
26                 && grid[nrow][ncol] == '1' && !vis[nrow][ncol]) {
27                     vis[nrow][ncol] = 1;
28                     q.push({nrow, ncol});
29                 }
30             }
31         }
32     }
33 }
34 public:
35     // Function to find the number of islands.
36     int numIslands(vector<vector<char>>& grid) {
37         int n = grid.size();
38         int m = grid[0].size();
39         vector<vector<int>> vis(n, vector<int>(m, 0));
40         int cnt = 0;
41         for(int row = 0; row < n; row++) {
42             for(int col = 0; col < m; col++) {
43                 if(!vis[row][col] && grid[row][col] == '1') {
44                     cnt++;
45                     bfs(row, col, vis, grid);
46                 }
47             }
48         }
49         return cnt;
    }
```



Compile & Run